Multi – level Linked List

–Destroy List(Main and Child)

– Space Complexity and Algorithm

Program:

```
void destroyNodeRecursively(Node *node)
{
    if (node == nullptr)
        return;

    // 1. Destroy Child List first
    destroyNodeRecursively(node->child);

    // 2. Destroy Next List
    destroyNodeRecursively(node->next);

    // 3. Free current
    cout << "Freeing node: " << node->data << endl;
    free(node);
}

void destroyList()
{
    destroyNodeRecursively(head);
    head = nullptr;
    cout << "Entire list destroyed." << endl;
}
int main()
{
    int choice;
    while (true)
    {
        cout << "\n--- Multilevel List Menu ---" << endl;
        cout << "8. Destroy List" << endl;
       cin >> choice;
```

```
        switch (choice)
        {
       case 8:
            destroyList();
            break;
       default:
            cout << "Invalid choice" << endl;
        }
    }
    return 0;
}
```

Space Complexity

The Call Stack will always grow exactly as tall as the longest path from the root to the furthest leaf. Mathematically, Space Complexity $= O(H)$, where H is the Maximum Height of the structure.

Input Space Complexity

1. *Parameter – Only Space Complexity*

This measures strictly the memory consumed by the actual variables passed into the function's signature.

```
void destroyNodeRecursively(Node *node)
```

- ***The only parameter being passed is a single memory address (a pointer).***
- ***Regardless of whether that pointer leads to a single node, a balanced tree of 1,000 nodes, or a straight line of a million nodes, the variable node itself only ever takes up exactly one standard unit of memory (typically 8 bytes on a modern 64 – bit operating system).***

- ***Because the size of the parameter does not grow as the problem size*** (N) ***grows,***
  the Parameter − Only Space Complexity is exactly:

$$\boldsymbol{\Theta(1)}$$

2. *Full Input Space* (*Holistic Space Complexity*)

This measures the total physical RAM occupied by the entire data structure

that the algorithm is being asked to process, before the algorithm

even begins running.

To calculate this, we look at the physical blueprint of the data structure.

Based on your code, every single *Node* ***in memory consists of at least three things***:

1. *data* $(e.g., \textbf{an integer or string})$
2. *Node* $*$ *child* $(\textbf{a pointer})$
3. *Node* $*$ *next* $(\textbf{a pointer})$

Let's assume this single node struct takes up a constant amount of memory, which we will call C_{node} **.**

- ***If we want to destroy a structure containing N total nodes, the operating system had to previously allocate exactly N of these structs in the heap.***
- $\textbf{Total memory used by the data structure} = N \times C_{node}$ **.**
- ***Dropping the constant gives us exactly***:

$$\boldsymbol{\Theta(N)}$$

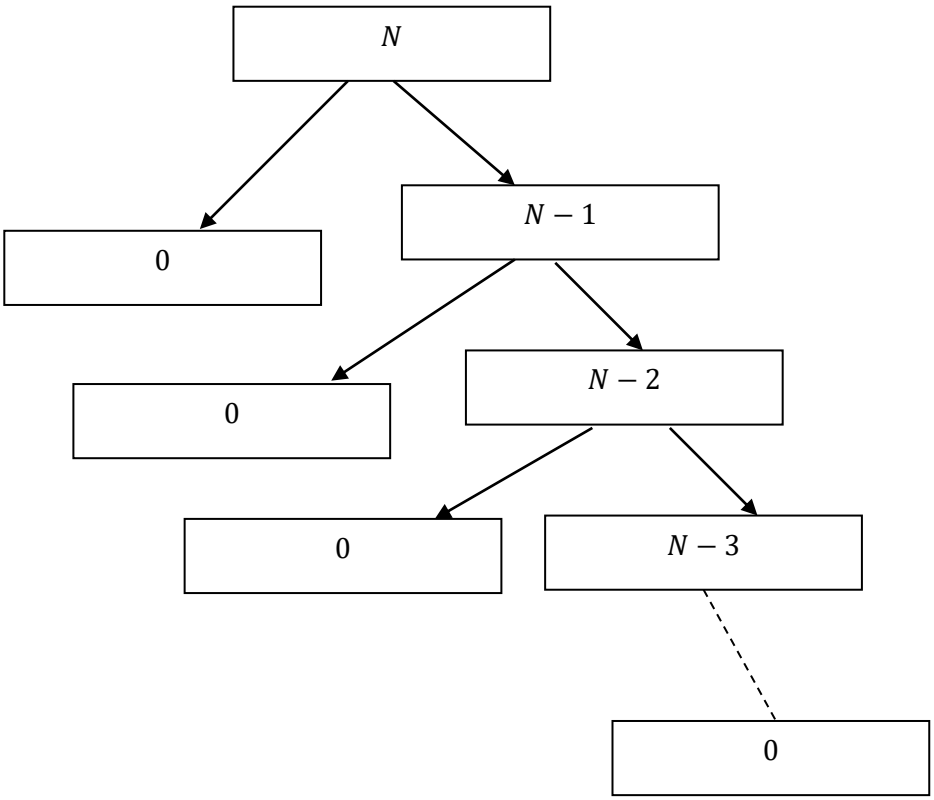
Auxiliary Space

Auxiliary space $=$ ***extra memory used during execution*** (***excluding input storage***).

Each stack frame stores:

1. ***Parameter***$(Node * node)$
2. ***The Return Address***.
3. ***The Base Pointer or Saved Registers***.
4. ***Local Variables***.

A. ***For Unbalanced Tree***(***Worst Structure***):



$\boldsymbol{Level}$ (k)	$\boldsymbol{No.of\ problems}$	$\boldsymbol{Problem\ Size}$	$\boldsymbol{Work\ /Cost\ Per\ Problem\ (Print\ and\ Add)}$	$\boldsymbol{Work\ done = Work\ per\ Problem \times No.of\ Problems}$
$\mathbf{0}$	$\mathbf{1}$	$\boldsymbol{n-0=n}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
$\mathbf{1}$	$\mathbf{1}$	$\boldsymbol{n-1}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
$\mathbf{2}$	$\mathbf{1}$	$\boldsymbol{n-2}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
. .	. .	. .	. .	. .
$\boldsymbol{n-1}$	$\mathbf{1}$	$\boldsymbol{n-(n-1)=1}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
$\boldsymbol{n}$ $\boldsymbol{(Base\ Case)}$	$\mathbf{1}$	$\boldsymbol{n-n=0}$ $\boldsymbol{(Base\ Case)}$	$\boldsymbol{T(0)}$	$\boldsymbol{1\times T(0)=T(0)}$

$\boldsymbol{We\ know, Edges = Nodes - 1}$

Here is the step − by − step mathematical breakdown:

1. ***Level* 0 (*The Root*)**: *The tree starts at the very top with the initial input value of* n.

2. ***Level* 1**: *The algorithm branches, and* 1 *is subtracted from the input. The nodes here hold the value* $n - 1$.

3. ***Level* 2**: *The algorithm branches again, subtracting another* 1. *The nodes hold the value* $n - 2$.

4. ***Level k* (*The Leaves*)**: *This pattern of subtracting* 1 *continues downwards until we reach the bottom of the tree (the base cases), where the nodes hold the value* 0. *Let's call this final depth level* k.

To find the actual depth (k), ***we set the formula for the value at level*** k ***equal to the value of the base case*** $[F(0)]$:

$$n - k = 0$$

Solving for k:

$$k = n$$

Conclusion:

The depth of the tree (the number of edges or steps from the root down to the farthest leaf) is exactly n.

As $k = n$, ***Levels goes from*** 0 ***to*** n.

Therefore , total nodes for level $k =$

$\Rightarrow 1 + (1 + 1 + \cdots$ `n` ***times***$)$.

$\Rightarrow 1 + n$ ***nodes***.

$\therefore$ ***Entire recursion tree will have*** $(1 + n) - 1 = n$ ***edges***.

For Depth Calculation

$$\boldsymbol{But\ for\ depth, we\ take\ edge(k) = n.}$$

$$\boldsymbol{N(Nodes) = Edges(k) + 1(Root)}$$

$$\boldsymbol{\Longrightarrow N = n + 1}\ ,$$

$$\boldsymbol{Maximum\ Depth = Number\ of\ Edges = n.}$$

$\boldsymbol{Based\ on\ Active\ stackframe}$:

$$\boldsymbol{\therefore Total\ Auxiliary\ Space}$$
$$= (\ \boldsymbol{Maximum\ Depth} + 1) \times \begin{pmatrix} \boldsymbol{Space\ per} \\ \boldsymbol{Recursive\ Call} \end{pmatrix}$$

$$\begin{bmatrix} \boldsymbol{Here + 1\ for\ Root\ Node\ , which\ is\ also\ an\ active} \\ \boldsymbol{stack\ frame} \end{bmatrix}$$

$$= \boldsymbol{\Theta}(\mathbf{n} + \mathbf{1}) \times \boldsymbol{\Theta}(\mathbf{1})$$

$$= \boldsymbol{\Theta}(\mathbf{n}) \times \boldsymbol{\Theta}(\mathbf{1})$$

$$= \boldsymbol{\Theta}(\boldsymbol{n})$$

$B.$ ***Balanced Structure***

$$\boldsymbol{2T\left(\frac{N-1}{2}\right)+C\,,\,Base\ Case:T(0)=1.}$$

Level (k)	***No. of problems***	***Problem Size***	***Work /Cost Per Problem (Print and free memory)***	***Work done = Work per Problem × No. of Problems***
$\mathbf{0}$	$\mathbf{1}$	n	C	$1\times C=C$
$\mathbf{1}$	2	$\frac{n-1}{2}$	C	$2\times C=2C$
$\mathbf{2}$	4	$\frac{n-3}{4}$	C	$4\times C=4C$
$\mathbf{3}$	$\mathbf{8}$	$\frac{n-7}{8}$	C	$8\times C=8C$
. .	. .	. .	. .	. .
k	2^k	$\frac{N-(2^k-1)}{2^k}$	C	$2^k\times C=2^kC$
$(Base\ Case)$	$N+1$	$n-n=0$ $(Base\ Case)$	$T(0)$	$(N+1)\times T(0)$

***Now we have to derive level `k` first*:**

$$\textbf{Problem size} = \frac{N - \left(2^k - 1\right)}{2^k}$$

$$\textbf{And we know, } T(0) \textbf{ is our base case .}$$

***The tree stops growing (it hits the base case)when the problem size reaches* 0.**

***So, to find the final level k, we set our pattern equal to* 0 *and solve for k*:**

$$\frac{N - \left(2^k - 1\right)}{2^k} = 0$$

***Multiplying* 2^k *on both the sides*:**

$$\Longrightarrow N - \left(2^k - 1\right) = 0$$

***Moving the* $2^k - 1$ *to the other side, we get*:**

$$\Longrightarrow N = 2^k - 1$$

***Add* 1 *to both sides to isolate the exponential term*:**

$$\Longrightarrow N + 1 = 2^k$$

***Take the base* $-$ 2 *logarithm of both sides*.**

***Because the base of our exponent is* $2(from\ 2^k)$*, we use the base* $-$ 2 *logarithm* (log_2)*to target it*.**

$$\Longrightarrow \log_2(N + 1) = \log_2 2^k$$

$$\Longrightarrow \log_2(N+1) = \mathrm{k} \times \log_2 2$$

$$\Longrightarrow \log_2(N+1) = k \; [\log_a a = 1]$$

$$\therefore k = \log_2(N+1)$$

Hence full table is:

Level (k)	***No. of problems***	***Problem Size***	***Work /Cost Per Problem (Print and free memory)***	***Work done = Work per Problem × No. of Problems***
0	1	n	C	$1 \times C = C$
1	2	$\frac{n-1}{2}$	C	$2 \times C = 2C$
2	4	$\frac{n-3}{4}$	C	$4 \times C = 4C$
3	8	$\frac{n-7}{8}$	C	$8 \times C = 8C$
. .	. .	. .	. .	. .
k	2^k	$\frac{N-(2^k-1)}{2^k}$	C	$2^k \times C = 2^k C$
. . .	. . .	. . .	. . .	. . .
$\log_2(N+1)$ $(Base\ Case)$	$N+1$	$n-n=0$ $(Base\ Case)$	$T(0)$	$(N+1) \times T(0)$

$$Hence\ , Number\ of\ levels : 0,1,2, \dots , k, \dots , \log_2(N+1)$$

$\boldsymbol{Type\ 1 : Calculation\ of\ total\ number\ of\ Nodes - Considering\ upto\ Level\ `k`\ :}$

$$\boldsymbol{Level\ 0 : 1\ node : 2^0}$$

$$\boldsymbol{Level\ 2 : 2\ node : 2^1}$$

$$\boldsymbol{Level\ 3 : 4\ node : 2^2}$$

.

.

$$\boldsymbol{Level\ k : 2^k\ node}$$

$$\boldsymbol{Total\ Nodes: 2^0 + 2^2 + \cdots + 2^k\ nodes}$$

$\boldsymbol{Applying\ geometric\ progression(series)}$:

$$\boldsymbol{S(n) = ar^0 + ar^1 + ar^2 + \cdots + ar^n = a\left(\frac{r^{n+1}-1}{r-1}\right), where}$$

$\boldsymbol{S(n)\ is\ sum\ of\ first\ n\ terms\ , a\ is\ coefficient\ and\ r\ is\ common\ ratio\ between\ the\ consecutive\ terms.}$

$$\Longrightarrow 1 \times 2^0 + 1 \times 2^2 + \cdots + 1 \times 2^k \; nodes$$

$$\Longrightarrow 1\left(\frac{2^{k+1} - 1}{2 - 1}\right)$$

$$\Longrightarrow 2^{k+1} - 1$$

$$\Longrightarrow 2^k \times 2 - 1$$

$$\therefore We\ know: 2^k = N + 1\ , applying\ we\ get:$$

$$\Longrightarrow (N + 1) \times 2 - 1$$

$$\Longrightarrow 2N + 2 - 1$$

$$\Longrightarrow 2N + 1$$

$Type\ 2 : Calculation\ of\ total\ number\ of\ Nodes - Considering\ upto\ Level\ `\log_2(N + 1)\ `:$

$$Level\ 0 : 1\ node : 2^0$$

$$Level\ 2 : 2\ node : 2^1$$

$$Level\ 3 : 4\ node : 2^2$$

.

.

$$Level\ k : 2^k\ node$$

.

.

$$\textbf{Level } \log_2(N+1) : 2^{\log_2(N+1)} \textbf{ node}$$

$$\textbf{Total Nodes}: 2^0 + 2^1 + 2^2 + \cdots + 2^{\log_2(N+1)} \textbf{ nodes}$$

Applying geometric progression(series) :

$$S(n) = ar^0 + ar^1 + ar^2 + \cdots + ar^n = a\left(\frac{r^{n+1}-1}{r-1}\right), \textbf{where}$$

$S(n)$ **is sum of first n terms ,** a **is coefficient and** r **is common ratio between the consecutive terms.**

$$\Longrightarrow 1 \times 2^0 + 1 \times 2^2 + \cdots + 1 \times 2^{\log_2(N+1)} \textbf{ nodes}$$

$$\Rightarrow 1\left(\frac{2^{\log_2(N+1)+1}-1}{2-1}\right)$$

$$\Rightarrow 2^{\log_2(N+1)+1} - 1$$

$$\Rightarrow 2 \times 2^{\log_2(N+1)} - 1$$

$$\Rightarrow 2 \times (N+1) - 1 \left[a^{\log_a b} = b\right]$$

$$\Longrightarrow 2N + 2 - 1$$

$$\Longrightarrow 2N + 1$$

$$\textbf{Total Edges} : \textbf{Nodes} - 1.$$

$$= 2N + 1 - 1$$

$$= 2N$$

The Total Edges $(2N)$ **represents every single jump the computer makes across the entire width of the tree.**
(This is why it maps to Time Complexity).

However, the Call Stack (Maximum Depth) does not care how wide the tree is. The stack only cares about the longest single, direct path from the root to the deepest leaf.

- **Total Edges** $= 2N$ **(Every line drawn in the entire tree)**
- $Maximum\ Depth = \log_2(N+1)$

$\left[(\textbf{Just the longest single path downward towards leaf})\right]$

Hence we don't need every line drawn in the entire tree.

From formula : $N(\textbf{Nodes}) = \textbf{Edges} + 1$ **, we can say**:

$$\Rightarrow \log_2(N+1) + 1$$

Let , $n = 3$

Imagine a tiny, perfectly balanced tree with exactly

3 real nodes $(1\ \textbf{Root}, 2\ \textbf{Children})$.

1. ***Calculate the edges*** **(*Depth*):**

$\log_2(N+1)$

$\Longrightarrow \log_2(3+1)$

$\Longrightarrow \log_2(4)$

$\Longrightarrow \log_2 2^2$

$\Longrightarrow 2$ ***jumps*** .

2. ***Trace the Execution Path*****:** ***Let's look at the actual stack frames when the computer dives down the left side*****:**

- ***Frame*** **1** **(*The Root*):** ***destroyNodeRecursively*****(*Root*)**

  (*Jump* 1)

- ***Frame*** **2** **(*The Child*):** ***destroyNodeRecursively*****(*Left_Child*)**

  (*Jump* 2)

- ***Frame*** **3** **(*The Base Case*):** ***destroyNodeRecursively*****(*nullptr*)**

The computer made exactly **2** ***jumps, which perfectly matches*** $\log_2(N+1)$. ***But there are*** **3** ***active stack frames sitting in memory, which perfectly matches*** $\log_2(N+1)+1$.

Hence , we will take all the active stack frames**:**

$$\therefore \textbf{Total Auxiliary Space} = (\textbf{Maximum Depth} + 1) \times \begin{pmatrix} \textbf{Space per} \\ \textbf{Recursive Call} \end{pmatrix}$$

Space per Recursive Call

Space per Recursive Call – represents the physical Stack Frame (or Activation Record) that the operating system creates every single time function calls itself.

In the case of destroyNodeRecursively function, the value that plugs into that multiplier is exactly $\Theta(1)$ (Constant Space).

1. What is inside that Space?

As we broke down earlier, every time the CPU jumps to a new recursive call, it carves out a small, fixed – size chunk of memory to keep track of where it is. For function, it holds:

- **The node pointer parameter (8 bytes)**
- **The Return Address (8 bytes)**
- **The Base/Frame pointers for system tracking (8 – 16 bytes)**
- **Local variables .**

Total size: Roughly ~24 to 32 bytes of physical RAM.

2.*Why is it* $\Theta(1)$?

Because those 24 bytes never change.

Whether you are destroying a tiny tree with 3 nodes or a massive tree with 1,000,000 nodes, the CPU always needs exactly ~24 bytes for a single stack frame.

Because the size of one frame does not grow as N grows, mathematically, we denote it as $\Theta(1)$.

.

$$= \Theta(\log_2(N+1)+1) \times \Theta(1)$$

$$= \big(\Theta(\log_2(N+1)) + \Theta(1)\big) \times \Theta(1)$$

$$= \Theta(\log_2(N+1))$$

$$If\ N \geq 1\ , then\ N+1 \leq N+N$$

Therefore:

$$N+1 \leq 2N$$

***Using $\log_2$ in both sides we get*:**

$$\log_2(N+1) \le \log_2 2N$$

$$\log_2(N+1) \le \log_2 2 + \log_2 N \quad \begin{pmatrix} \textit{Multiplication Rule}, \\ \log_b(x \cdot y) = \log_b x + \log_b y \end{pmatrix}$$

$$\log_2(N+1) \le 1 + \log_2 N$$

***For*, $N \ge 2$ *and* $\log_2 N \ge 1$.**

$$1 + \log_2 N \le \log_2 N + \log_2 N = 2\log_2 N$$

***Therefore , combining with the result*:**

$$\log_2(N+1) \le 1 + \log_2 N \le 2\log_2 N \text{ , } \textit{for all } N \ge 2$$

or*,** $\log_2(N+1) \le 2\log_2 N$, ***for all $N \ge 2$

The Big O definition : The definition of $f(N) = O\big(g(N)\big)$ ***is*:**
There exists constants $c > 0$ ***and*** n_0 ***such that***: $f(N) \le c \times g(N)$
for all $N \ge n_0$.

$if\ n_0 = 1\ , for\ all\ N \geq 1$

$$\log_2(N+1) \leq 1 + \log_2 N$$

$if\ n_0 = 2\ , for\ all\ N \geq 2$

$$\log_2(N+1) \leq 1 + \log_2 N \leq 2\log_2 N$$

$$or, \log_2(N+1) \leq 2\log_2 N$$

- $f(n) = log_2(N+1)$
- $g(n) = \log_2 N$

$if\ we\ take\ N \geq 1\ , then\ 1 + \log_2 N\ doesn't\ satisfy$

$f(N) \leq c \times g(N)\ definition\ , i.e.\ isn't\ of\ the\ form$

$c \times \log_2 N.$

$Also, lets\ see\ whether\ some\ constant\ `c`\ can\ satisfy:$

$$1 + \log_2 N \leq c \times \log_2 N\ [c \times g(n), where\ , g(n) = \log_2 N]$$

$For\ N \geq n_0\ or, N \geq 1\ , Lets\ take\ N = 1:$

$$\Longrightarrow 1 + \log_2(1) \leq c \times \log_2(1)$$

$$\Rightarrow 1 + 0 \leq c \times 0 \; [\log_2 1 = 0]$$

$$\Rightarrow 1 \leq 0$$

which is impossible.

Therefore no finite constant `c`can make the inequality work starting at $N = 1$.

if we take $N \geq 2$, ***then*** $2 \log_2 N$ ***does satisfy*** $f(N) \leq c \times g(N)$ ***definition , i.e. is in the form of*** $c \times \log_2 N$.

Also, lets see whether some constant `c` can satisfy:

$$2 \log_2 N \leq c \times \log_2 N \; [c \times g(n), where \; , g(n) = \log_2 N]$$

For $N \geq n_0$ ***or,*** $N \geq 2$, ***Lets take*** $N = 2$:

$$\Rightarrow 2 \log_2 2 \leq c \times \log_2 2$$

$$\Rightarrow 2 \leq c \times 1 \; [\log_2 2 = 1]$$

which is very much possible.

Therefore,

$$\log_2(N+1) \leq 2\log_2 N$$

which is exactly the required form:

$$f(N) \leq c \times g(N), for\ all\ N \geq n_0.$$

with:

$$c = 2\ , n_0 = 2$$

Thats why we conclude:

$$\log_2(N+1) = O(2\log_2 N) = O(\log_2 N)$$
$$i.e. \log_2(N+1) = O(\log_2 N)$$

In fact, since

$$\log_2 N \leq \log_2(N+1)$$

$$i.e. \log_2(N+1) = \Omega(\log_2 N)$$

We also have :

$$\log_2(N+1) = \Theta(\log_2 N)[As\ both\ \Omega\ and\ O\ exists.]$$

***Algorithm*: *DestroyList*()**

1. [***Initiate Recursion***] ***Call DestroyNodeRecursively*(*START*)**

– –

***Translate Call DestroyNodeRecursively*(*CURRENT*) :**

1. [***Base Case*; *Null Check***] ***if NODEPTR = NULL* , *Then*:**

Return and exit the current recursive call.

2. [***Destroy Child List***]***Call DestroyNodeRecursively*(*CHILD*[*CURRENT*])**

(*This traverses down the child pointers first*).

3. [***Destroy Next List***]***Call DestroyNodeRecursively*(*LINK*[*CURRENT*])**

(*This traverses down the next pointers*).

4. [***Free Current Node***]

- ***Write*:``*Freeing node*``, *INFO*[*CURRENT*].**
- ***Free*(*CURRENT*)**
  (Deallocate/Free the memory assigned to CURRENT.)

5. [Return] **Return to the previous caller.**

– –

2. [***Reset Head Pointer***] ***SET START* := *NULL*.**

3. [***Confirmation***]***Write*:``*Entire list destroyed*.``**

4. [***Exit***]***Exit the program or function*.**